

Homework #2

CSE 446/546: Machine Learning

Madeline Brown

Due: **Monday** November 04, 2024 11:59pm

A: 108 points, **B:** 20 points

Outside sources used: <https://www.youtube.com/watch?v=vMh0zPT0tLI&t=1s>
<https://www.youtube.com/watch?v=UmathvAKj80>
perplexity.ai for definitions and syntax

Please review all homework guidance posted on the website before submitting it to Gradescope. Reminders:

- All code must be written in Python and all written work must be typeset (e.g. $\text{\LaTeX}$).
- Make sure to read the “What to Submit” section following each question and include all items.
- Please provide succinct answers and supporting reasoning for each question. Similarly, when discussing experimental results, concisely create tables and/or figures when appropriate to organize the experimental results. All explanations, tables, and figures for any particular part of a question must be grouped together.
- For every problem involving generating plots, please include the plots as part of your PDF submission.
- When submitting to Gradescope, please link each question from the homework in Gradescope to the location of its answer in your homework PDF. Failure to do so may result in deductions of up to 10% of the value of each question not properly linked. For instructions, see https://www.gradescope.com/get_started#student-submission.

Not adhering to these reminders may result in point deductions.

Important: By turning in this assignment (and all that follow), you acknowledge that you have read and understood the collaboration policy with humans and AI assistants alike: <https://courses.cs.washington.edu/courses/cse446/24au/assignments/>. Any questions about the policy should be raised at least 24 hours before the assignment is due. There are no warnings or second chances. If we suspect you have violated the collaboration policy, we will report it to the college of engineering who will complete an investigation.

Not adhering to these reminders may result in point deductions.

Conceptual Questions

A1. The answers to these questions should be answerable without referring to external materials. Briefly justify your answers with a few words.

- a. [2 points] Suppose that your estimated model for predicting house prices has a large positive weight on the feature **number of bathrooms**. If we remove this feature and refit the model, will the new model have a strictly higher error than before? Why?

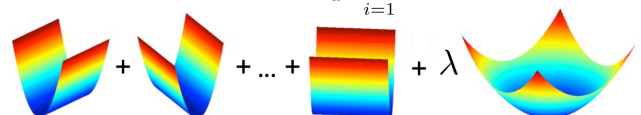
The new model with **number of bathrooms** removed will not necessarily have strictly higher error. It is possible that once we remove a seemingly significant feature that we will end up with a simpler model that is a better predictor. Specifically, it is possible that the large positive weight on **number of bathrooms** was due to noise or redundancy with other variables, so removing it could actually be a better reflection of the data and lead to less error, if that were true.

- b. [2 points] Compared to L2 norm penalty, explain why a L1 norm penalty is more likely to result in sparsity (a larger number of 0s) in the weight vector.

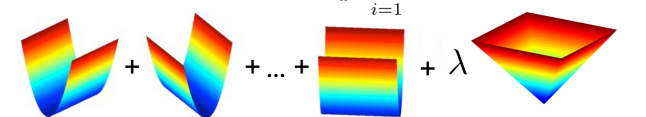
As we saw in lecture, the Lasso (L1) penalty introduces sharp corners into the objective. When we go to minimize this function, the sharp sides have constant slope, so in gradient descent, this tends to drive entries of the weight vector all the way down toward zero at a higher rate.

Ridge vs. Lasso Regression

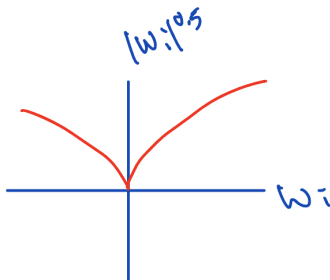
- Ridge Regression objective:

$$\hat{w}_{ridge} = \arg \min_w \sum_{i=1}^n (y_i - x_i^T w)^2 + \lambda \|w\|_2^2$$


- Lasso objective:

$$\hat{w}_{lasso} = \arg \min_w \sum_{i=1}^n (y_i - x_i^T w)^2 + \lambda \|w\|_1$$


- c. [2 points] In at most one sentence each, state one possible upside and one possible downside of using the following regularizer: $(\sum_i |w_i|^{0.5})$.



Since the gradient/slope of this regularizer has a higher magnitude for smaller weights (closer to 0), this would drive more weights close 0 but not exactly equal to 0, which could be a plus if we want to keep more features. One possible downside is that the function $|w_i|^{0.5}$ itself is not convex, which is a little scary and might make it harder to optimize in the context of what we have learned in lecture.

- d. [1 point] True or False: If the step-size for gradient descent is too large, it may not converge.

True. We saw this in lecture: if we start out relatively close to the minimum but have a step size too large, we might jump over the minimum at our next step. Then each step we would get further and further away from the minimum, and end up doing gradient ascent instead.

- e. [2 points] In your own words, describe why stochastic gradient descent (SGD) works, even though only a small portion of the data is considered at each update.

Even though SGD does not use all the data, it still uses enough of the data to get a good-enough approximation of the gradient at a given point, in terms of finding the optimal solution. By using less of the data it is less subject to outliers in the data, but is otherwise the same procedure as gradient descent, leading to a solution.

- f. [2 points] In at most one sentence each, state one possible advantage of SGD over GD (gradient descent), and one possible disadvantage of SGD relative to GD.

SGD may be faster in practice, since the individual step time is lower than for GD (according to the Murphy book).

Since SGD only uses some of the data during each update, it may give a less accurate answer, or end up settling on a non-optimal answer compared to GD.

What to Submit:

- **Part d:** True or False.
- **Parts a-f:** Brief (2-3 sentence) explanation.

Convexity and Norms

A2. A norm $\|\cdot\|$ over $\mathbb{R}^n$ is defined by the properties: (i) non-negativity: $\|x\| \geq 0$ for all $x \in \mathbb{R}^n$ with equality if and only if $x = 0$, (ii) absolute scalability: $\|ax\| = |a| \|x\|$ for all $a \in \mathbb{R}$ and $x \in \mathbb{R}^n$, (iii) triangle inequality: $\|x + y\| \leq \|x\| + \|y\|$ for all $x, y \in \mathbb{R}^n$.

- a. [3 points] Show that $f(x) = (\sum_{i=1}^n |x_i|)$ is a norm. (Hint: for (iii), begin by showing that $|a + b| \leq |a| + |b|$ for all $a, b \in \mathbb{R}$.)

Proof. Let $x, y \in \mathbb{R}^n$, and $a, b \in \mathbb{R}$. Then we have the following:

(i) $f(x) = (\sum_{i=1}^n |x_i|) \geq 0$ since $|x_i| \geq 0$ for any $x_i \in \mathbb{R}$ by definition. Additionally, if $x = 0$, then $f(0)$ is the sum of zero entries and is thus zero. If $\sum_{i=1}^n |x_i| = 0$, then since this is a sum of nonnegative terms, every term must be zero. So $f(x) = 0$ if and only if $x = 0$, and non-negativity is satisfied.

(ii) Note that the entries of ax are given by $(ax)_i = ax_i$ by definition. Then

$$f(ax) = \left(\sum_{i=1}^n |(ax)_i| \right) = \left(\sum_{i=1}^n |ax_i| \right) = \left(\sum_{i=1}^n |a| |x_i| \right) = \left(|a| \sum_{i=1}^n |x_i| \right) = |a| f(x),$$

where we factored $|a|$ out of the sum because it does not depend on i . So f satisfies absolute scalability.

(iii) Per the hint, we first show the triangle inequality for $a, b \in \mathbb{R}$. We have two cases:

If $a + b \geq 0$, then $a + b = |a + b|$ and $a + b \leq |a| + |b|$ since $a \leq |a|$ and $b \leq |b|$, so $|a + b| \leq |a| + |b|$, putting everything together.

If $a + b < 0$, then $|a + b| = -(a + b) = -a - b$. We also have $-a - b \leq |a| + |b|$, since $-a \leq |a|$ and $-b \leq |b|$. So $|a + b| \leq |a| + |b|$.

Now note that $(x + y)_i = x_i + y_i$, so by the above, $|(x + y)_i| = |x_i + y_i| \leq |x_i| + |y_i|$. Then

$$f(x + y) = \sum_{i=1}^n |(x + y)_i| = \sum_{i=1}^n |x_i + y_i| \leq \sum_{i=1}^n (|x_i| + |y_i|) = \sum_{i=1}^n |x_i| + \sum_{i=1}^n |y_i| = f(x) + f(y),$$

proving the triangle inequality. $\square$

- b. [2 points] Show that $g(x) = (\sum_{i=1}^n |x_i|^{1/2})^2$ is not a norm. (Hint: it suffices to find two points in $n = 2$ dimensions such that the triangle inequality does not hold.)

Proof. Per the hint, we provide a contradiction. Consider vectors in $\mathbb{R}^2$, $x = \begin{bmatrix} 3 \\ 0 \end{bmatrix}$ and $y = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$ so that

$x + y = \begin{bmatrix} 3 \\ 1 \end{bmatrix}$. Then

$$g(x) = (3^{1/2} + 0)^2 = 3,$$

and

$$g(y) = (0 + 1^{1/2})^2 = 1,$$

but

$$g(x + y) = (3^{1/2} + 1^{1/2})^2 \approx 7.464.$$

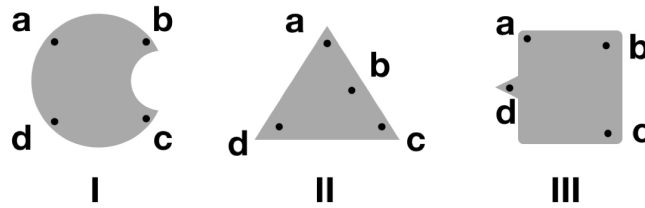
Then $g(x + y) > g(x) + g(y)$, so g does not satisfy the triangle inequality and thus it is not a norm. $\square$

Context: norms are often used in regularization to encourage specific behaviors of solutions. If we define $\|x\|_p := (\sum_{i=1}^n |x_i|^p)^{1/p}$ then one can show that $\|x\|_p$ is a norm for all $p \geq 1$. The important cases of $p = 2$ and $p = 1$ correspond to the penalty for ridge regression and the lasso, respectively.

What to Submit:

- **Parts a, b:** Proof.

A3. [3 points] A set $A \subseteq \mathbb{R}^n$ is *convex* if $\lambda x + (1 - \lambda)y \in A$ for all $x, y \in A$ and $\lambda \in [0, 1]$. For each of the grey-shaded sets below (I-III), state whether each one is convex, or state why it is not convex using any of the points a, b, c, d in your answer.

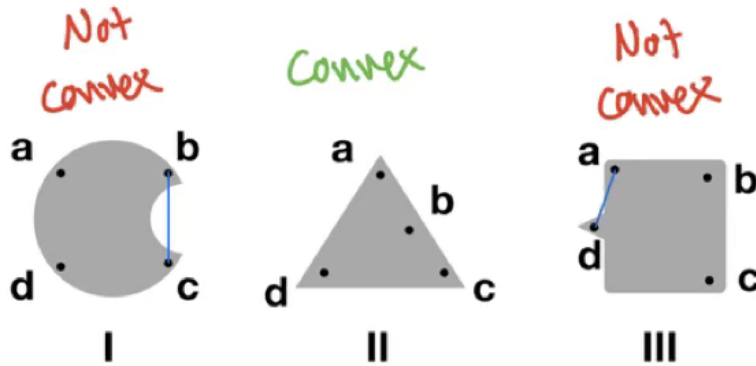


We interpret the given definition of convex as meaning that a straight line connecting two points in our set must also itself stay in the set, otherwise the set is not convex.

I is not convex, as a line connecting b and c will exit the set.

II is convex, this is an example we saw in class but a line connecting any two points within the triangle will stay within the triangle.

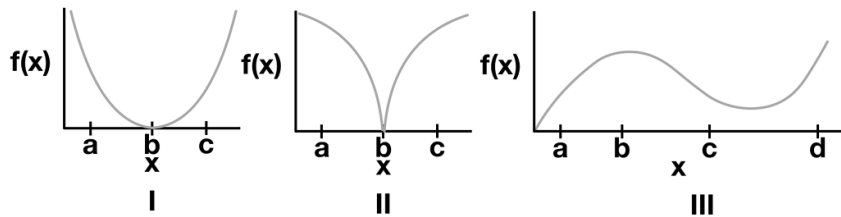
III is not convex, as a line connecting a and d would exit the set.



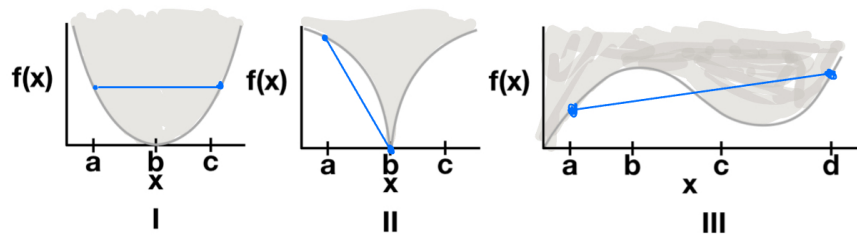
What to Submit:

- **Parts I-III:** 1-2 sentence explanation of why the diagram is convex or not.

A4. [4 points] We say a function $f : \mathbb{R}^d \rightarrow \mathbb{R}$ is convex on a set A if $f(\lambda x + (1 - \lambda)y) \leq \lambda f(x) + (1 - \lambda)f(y)$ for all $x, y \in A$ and $\lambda \in [0, 1]$. For each of the functions shown below (I-III), state whether each is convex on the specified interval, or state why not with a counterexample using any of the points a, b, c, d in your answer.



For these functions to be convex, a line connecting any two points on the function in the specified interval must stay above the function, as according to the definition given. We can similarly think of this like problem A3 above, where the functions form the lower portion of a set, and we need a line connecting any two points in the lower boundary of the “set” to stay within that set (the gray shaded portions on the below figure).



a. Function in panel I on $[a, c]$

This is convex because it is curved downward, so a line connecting any two points will stay above the function.

b. Function in panel II on $[a, c]$

This is not convex, as a line connecting a and $b \in [a, c]$ goes below the function/exits the “set.”

c. Function in panel III on $[a, d]$

This is not convex, as a line connecting a and d goes below the function.

d. Function in panel III on $[c, d]$

This is convex, similar to (a) it has the shape of a U so connecting any two points on $f(x)$ in $[c, d]$ will stay above $f(x)$.

What to Submit:

- **Parts a-d:** 1-2 sentence explanation of why the function is convex or not.

B1. Use just the definitions above and let $\|\cdot\|$ be a norm.

- a. [3 points] Show that $f(x) = \|x\|$ is a convex function.
- b. [3 points] Show that $\{x \in \mathbb{R}^n : \|x\| \leq 1\}$ is a convex set.
- c. [2 points] Draw a picture of the set $\{(x_1, x_2) : g(x_1, x_2) \leq 4\}$ where $g(x_1, x_2) = (|x_1|^{1/2} + |x_2|^{1/2})^2$. (This is the function considered in 1b above specialized to $n = 2$.) We know g is not a norm. Is the defined set convex? Why not?

Context: It is a fact that a function f defined over a set $A \subseteq \mathbb{R}^n$ is convex if and only if the set $\{(x, z) \in \mathbb{R}^{n+1} : z \geq f(x), x \in A\}$ is convex. Draw a picture of this for yourself to be sure you understand it.

What to Submit:

- **Parts a, b:** Proof.
- **Part c:** A picture of the set, and 1-2 sentence explanation.

Lasso on a Real Dataset

Given $\lambda > 0$ and data $(x_i, y_i)_{i=1}^n$, the Lasso is the problem of solving

$$\arg \min_{w \in \mathbb{R}^d, b \in \mathbb{R}} \sum_{i=1}^n (x_i^T w + b - y_i)^2 + \lambda \sum_{j=1}^d |w_j|$$

where λ is a regularization parameter. For the programming part of this homework, you will implement the coordinate descent method shown in Algorithm to solve the Lasso problem. This is a variant of the subgradient descent method and a more detailed discussion can be found in these [slides](#).

You may use common computing packages (such as `numpy` or `scipy`), but do not use an existing Lasso solver (e.g., of `scikit-learn`).

[h] this is the wrong algorithm I just didn't copy the new one in here not converged $b \leftarrow \frac{1}{n} \sum_{i=1}^n \left(y_i - \sum_{j=1}^d w_j x_{i,j} \right)$

$k \in \{1, 2, \dots, d\} \quad a_k \leftarrow 2 \sum_{i=1}^n x_{i,k}^2$

$c_k \leftarrow 2 \sum_{i=1}^n x_{i,k} \left(y_i - (b + \sum_{j \neq k} w_j x_{i,j}) \right)$

$$w_k \leftarrow \begin{cases} (c_k + \lambda)/a_k & c_k < -\lambda \\ 0 & c_k \in [-\lambda, \lambda] \\ (c_k - \lambda)/a_k & c_k > \lambda \end{cases}$$

Before you get started, the following hints may be useful:

- Wherever possible, use matrix libraries for matrix operations (not `for` loops).
- There are opportunities to considerably speed up parts of the algorithm by precomputing quantities like a_k before the `for` loop; you are permitted to add these improvements (and it may save you some time).
- As a sanity check, ensure the objective value is nonincreasing with each step.
- It is up to you to decide on a suitable stopping condition. A common criteria is to stop when no element of w changes by more than some small δ during an iteration. If you need your algorithm to run faster, an easy place to start is to loosen this condition.
- You will need to solve the Lasso on the same dataset for many values of λ . This is called a regularization path. One way to do this efficiently is to start at a large λ , and then for each consecutive solution, initialize the algorithm with the previous solution, decreasing λ by a constant ratio (e.g., by a factor of 2).
- The smallest value of λ for which the solution $\hat{w}$ is entirely zero is given by

$$\lambda_{max} = \max_{k=1, \dots, d} 2 \left| \sum_{i=1}^n x_{i,k} \left(y_i - \left(\frac{1}{n} \sum_{j=1}^n y_j \right) \right) \right| \quad (1)$$

This is helpful for choosing the first λ in a regularization path.

A5. We will first try out your solver with some synthetic data. A benefit of the Lasso is that if we believe many features are irrelevant for predicting y , the Lasso can be used to enforce a sparse solution, effectively differentiating between the relevant and irrelevant features. Suppose that $x \in \mathbb{R}^d, y \in \mathbb{R}, k < d$, and data are generated independently according to the model $y_i = w^T x_i + \epsilon_i$ where

$$w_j = \begin{cases} j/k & \text{if } j \in \{1, \dots, k\} \\ 0 & \text{otherwise} \end{cases} \quad (2)$$

and $\epsilon_i \sim \mathcal{N}(0, \sigma^2)$ is noise (note that in the model above $b = 0$). We can see from Equation (2) that since $k < d$ and $w_j = 0$ for $j > k$, the features $k + 1$ through d are irrelevant for predicting y .

Generate a dataset using this model with $n = 500, d = 1000, k = 100$, and $\sigma = 1$. You should generate the dataset such that each $\epsilon_i \sim \mathcal{N}(0, 1)$, and y_i is generated as specified above. You are free to choose a distribution from which the x 's are drawn, but make sure standardize the x 's before running your experiments.

- a. [10 points] With your synthetic data, solve multiple Lasso problems on a regularization path, starting at λ_{max} where no features are selected (see Equation (1)) and decreasing λ by a constant ratio (e.g., 2) until nearly all the features are chosen. In plot 1, plot the number of non-zeros as a function of λ on the x-axis (Tip: use `plt.xscale('log')`).

Below, I have chosen to generate the data with x 's being standard Gaussian random variables.

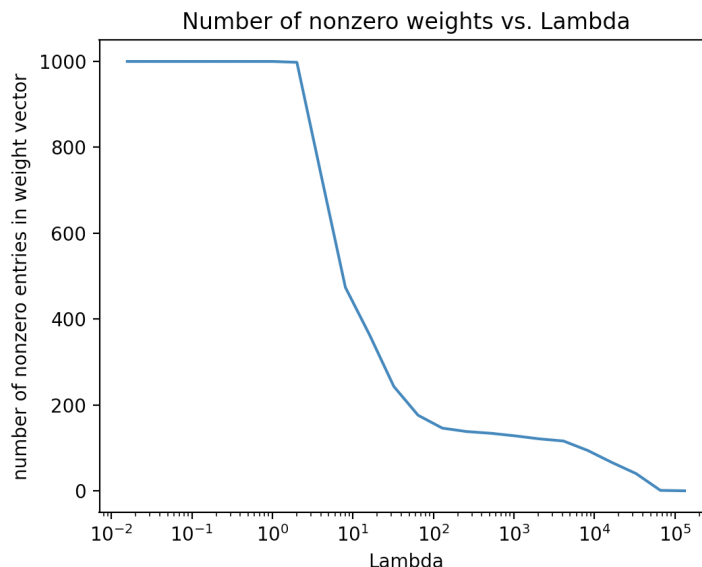


Figure 1: Number of non-zeros in weight vector as a function of λ on the x-axis

- b. [10 points] For each value of λ tried, record values for false discovery rate (FDR) (number of incorrect nonzeros in $\hat{w}$ /total number of nonzeros in $\hat{w}$) and true positive rate (TPR) (number of correct nonzeros in $\hat{w}/k$). Note: for each j , $\hat{w}_j$ is an incorrect nonzero if and only if $\hat{w}_j \neq 0$ while $w_j = 0$. In plot 2, plot these values with the x-axis as FDR, and the y-axis as TPR.

Note that in an ideal situation we would have an (FDR,TPR) pair in the upper left corner. We can always trivially achieve (0,0) and $(\frac{d-k}{d}, 1)$.

- c. [5 points] Comment on the effect of λ in these two plots in 1-2 sentences.

In plot 1, we see that as λ increases, the number of nonzero entries in the weight vector decreases, which means that there are more zero entries in the weight vector, consistent with our idea that more regularization would lead to a more sparse weight vector.

In plot 2, we see generally that as the FDR increases, so does the TPR, but the graph flattens out as it increases. A larger λ will lead to a higher FDR because there will be more sparsity in the weight vector,

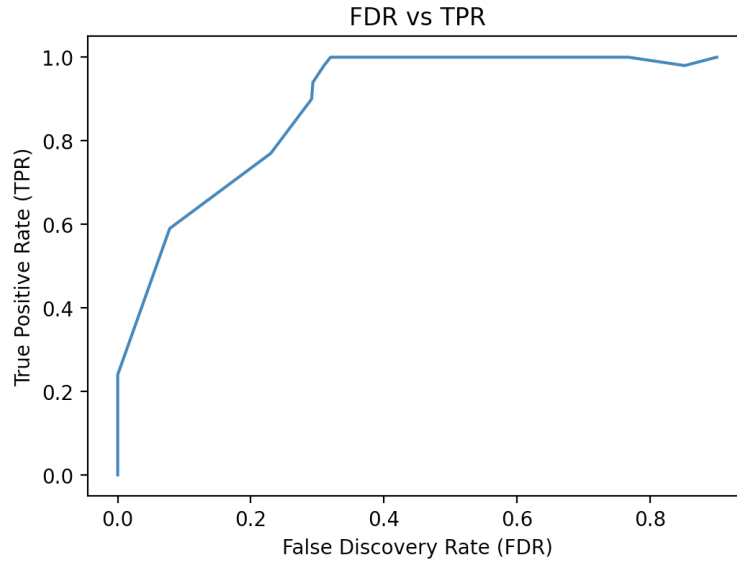


Figure 2: False Discovery Rate (FDR) on the x axis and True Positive Rate (TPR) on the y axis, calculated for varying λ values.

so there would be more “incorrect” nonzeros. The TPR is larger with larger λ also because of the sparsity, leading to fewer nonzero entries overall. As the note in part (b) says, when λ goes to 0, the (FDR, TPR) pair goes to $(\frac{d-k}{d}, 1)$.

What to Submit:

- **Part a:** Plot 1.
- **Part b:** Plot 2.
- **Part c:** 1-2 sentence explanation.
- **Code** on Gradescope through coding submission

A6. We'll now put the Lasso to work on some real data in `crime_data_lasso.py`. We have read in the data for you with the following:

```
df_train, df_test = load_dataset("crime")
```

This stores the data as Pandas `DataFrame` objects. `DataFrames` are similar to Numpy `arrays` but more flexible; unlike `arrays`, `DataFrames` store row and column indices along with the values of the data. Each column of a `DataFrame` can also store data of a different type (here, all data are floats). Here are a few commands that will get you working with Pandas for this assignment:

```
df.head()           # Print the first few lines of DataFrame df.
df.index            # Get the row indices for df.
df.columns          # Get the column indices.
df['foo']           # Return the column named 'foo'.
df.drop('foo', axis = 1) # Return all columns except 'foo'.
df.values           # Return the values as a Numpy array.
df['foo'].values     # Grab column foo and convert to Numpy array.
df.iloc[:3,:3]      # Use numerical indices (like Numpy) to get 3 rows and cols.
```

The data consist of local crime statistics for 1,994 US communities. The response y is the rate of violent crimes reported per capita in a community. The name of the response variable is `ViolentCrimesPerPop`, and it is held in the first column of `df_train` and `df_test`. There are 95 features. These features include many variables. Some features are the consequence of complex political processes, such as the size of the police force and other systemic and historical factors. Others are demographic characteristics of the community, including self-reported statistics about race, age, education, and employment drawn from Census reports.

The goals of this problem are threefold: (i) to encourage you to think about how data collection processes affect the resulting model trained from that data; (ii) to encourage you to think deeply about models you might train and how they might be misused; and (iii) to see how Lasso encourages sparsity of linear models in settings where d is large relative to n . We emphasize that training a model on this dataset can suggest a degree of correlation between a community's demographics and the rate at which a community experiences and reports violent crime. We strongly encourage students to consider why these correlations may or may not hold more generally, whether correlations might result from a common cause, and what issues can result in misinterpreting what a model can explain.

The dataset is split into a training and test set with 1,595 and 399 entries, respectively¹. We will use this training set to fit a model to predict the crime rate in new communities and evaluate model performance on the test set. As there are a considerable number of input variables and fairly few training observations, overfitting is a serious issue. In order to avoid this, use the coordinate descent Lasso algorithm implemented in the previous problem.

¹The features have been standardized to have mean 0 and variance 1.

- a. [4 points] Read the documentation for the original version of this dataset: <http://archive.ics.uci.edu/ml/datasets/communities+and+crime>. Report 3 features included in this dataset for which historical *policy* choices in the US would lead to variability in these features. As an example, the *number of police* in a community is often the consequence of decisions made by governing bodies, elections, and amount of tax revenue available to decision makers.

Any of the race features (racePctBlack, racePctAsian, etc.) would have variability from historical US policy choices due to historical housing discrimination based on race, or redlining. PctVacantBoarded may depend on policy, assuming there are different laws in different areas that dictate the conditions under which a vacant house is boarded up, so that may vary by area. NumInShelters also depends on the number of shelters available in an area, which may vary by policymaker.

- b. [4 points] Before you train a model, describe 3 features in the dataset which might, if found to have nonzero weight in model, be interpreted as *reasons* for higher levels of violent crime, but which might actually be a *result* rather than (or in addition to being) the cause of this violence.

Anything regarding unhoused people, such as NumStreet, may be a result of crime, as the people who committed the crimes may end up homeless as a result of their sentencing or fines, rather than the other way around. Lower rent (RentMedian, etc.) may be the result of crime as well due to fewer people not wanting to live in the area, as might PolicAveOTWorked, since police might work more after lots of crime is committed. Any of these could also be interpreted as a reason for more crime as well, as it could be (falsely) assumed that people with a lower/no income would have more of a “reason” to commit violent crimes.

Now, we will run the Lasso solver. Begin with $\lambda = \lambda_{\max}$ defined in Equation (1). Initialize all weights to 0. Then, reduce λ by a factor of 2 and run again, but this time initialize $\hat{w}$ from your $\lambda = \lambda_{\max}$ solution as your initial weights, as described above. Continue the process of reducing λ by a factor of 2 until $\lambda < 0.01$. For all plots use a log-scale for the λ dimension (Tip: use `plt.xscale('log')`).

- c. [4 points] Plot the number of nonzero weights of each solution as a function of λ .

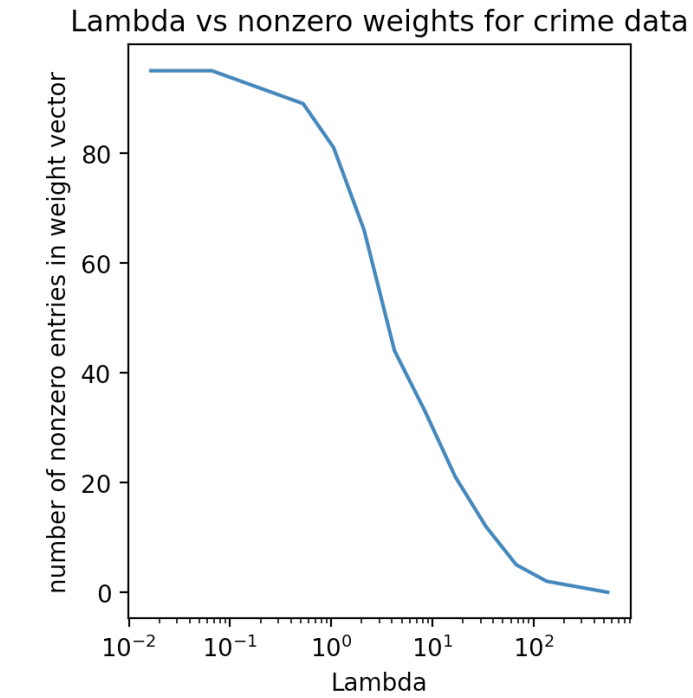


Figure 3: Plot 1 Number of zero entries in weight as a function of lambda for crime data.

- d. [4 points] Plot the regularization paths (in one plot) for the coefficients for input variables `agePct12t29`, `pctWSocSec`, `pctUrban`, `agePct65up`, and `householdsize`.

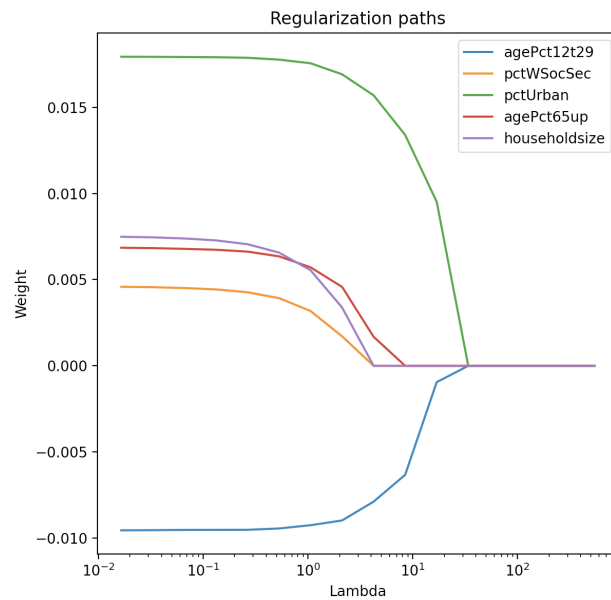


Figure 4: Plot 2 Regularization paths for the coefficients for input variables `agePct12t29`, `pctWSocSec`, `pctUrban`, `agePct65up`, and `householdsize`.

- e. [4 points] On one plot, plot the squared error on the training and test data as a function of λ .

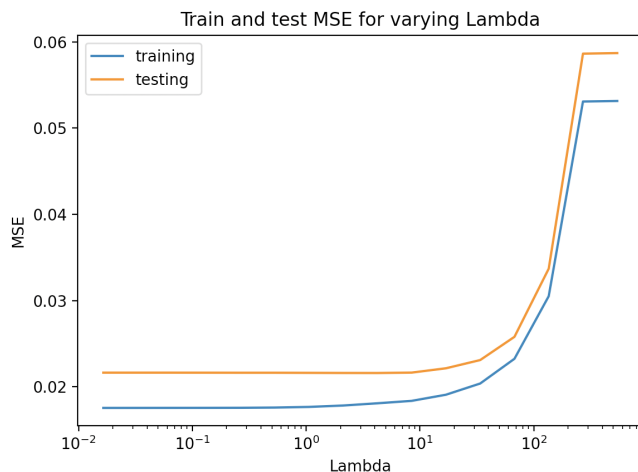


Figure 5: Plot 3 Squared error on the training and test data as a function of λ .

- f. [4 points] Sometimes a larger value of λ performs nearly as well as a smaller value, but a larger value will select fewer variables and perhaps be more interpretable. Retrain and inspect the weights $\hat{w}$ for $\lambda = 30$ and for *all* input variables. Which feature had the largest (most positive) Lasso coefficient? What about the most negative? Discuss briefly.

```
Most Positive Weight: PctIlleg: 0.0629994308447673
Most Negative Weight: PctKids2Par: -0.04029186488913935
```

Figure 6: Output screenshot of most positive and most negative weighted variables for $\lambda = 30$.

The feature with the largest/most positive weight is PctIlleg, which is the percentage of kids born to never married [parents], indicating that it has the largest contribution to violent crime. The variable with the most negative Lasso coefficient is PctKids2Par percentage of kids in family housing with two parents, implying that the more kids with two parents there are, the less violent crime there is. Since both of these have to do with status of families with children, this suggests a correlation between family makeup and violent crime.

- g. [4 points] Suppose there was a large negative weight on agePct65up and upon seeing this result, a politician suggests policies that encourage people over the age of 65 to move to high crime areas in an effort to reduce crime. What is the (statistical) flaw in this line of reasoning? (Hint: fire trucks are often seen around burning buildings, do fire trucks cause fire?)

The flaw is that the politician is mixing up correlation and causation. While people over the age of 65 might not be correlated to high crime areas, this does not mean that their presence alone (or at all) is preventing crime from occurring. There could be other reasons that agePct65up has a large negative weight, including the possibility that older people are less likely to commit violent crimes, or that older people skew toward more affluent areas, etc.

What to Submit:

- **Parts a, b:** 1-3 sentence explanation.
- **Part c:** Plot 1.
- **Part d:** Plot 2.
- **Part e:** Plot 3.

- **Parts f, g:** Answers and 1-2 sentence explanation.
- **Code** on Gradescope through coding submission.

Logistic Regression

Binary Logistic Regression

A7. Here we consider the MNIST dataset, but for binary classification. Specifically, the task is to determine whether a digit is a 2 or 7. Here, let $Y = 1$ for all the “7” digits in the dataset, and use $Y = -1$ for “2”. We will use regularized logistic regression. Given a binary classification dataset $\{(x_i, y_i)\}_{i=1}^n$ for $x_i \in \mathbb{R}^d$ and $y_i \in \{-1, 1\}$ we showed in class that the regularized negative log likelihood objective function can be written as

$$J(w, b) = \frac{1}{n} \sum_{i=1}^n \log(1 + \exp(-y_i(b + x_i^T w))) + \lambda \|w\|_2^2$$

Note that the offset term b is not regularized. For all experiments, use $\lambda = 10^{-1}$. Let $\mu_i(w, b) = \frac{1}{1 + \exp(-y_i(b + x_i^T w))}$.

- a. [8 points] Derive the gradients $\nabla_w J(w, b)$, $\nabla_b J(w, b)$ and give your answers in terms of $\mu_i(w, b)$ (your answers should not contain exponentials).

Claim: $\nabla_w J(w, b) = \frac{1}{n} \sum_{i=1}^n -y_i x_i (1 - \mu_i(w, b)) + 2\lambda w$.

Proof. By chain rule and our gradient rules, and noting that y_i is a scalar, we have

$$\nabla_w J(w, b) = \frac{1}{n} \sum_{i=1}^n \frac{1}{1 + \exp(-y_i(b + x_i^T w))} (-y_i x_i) \exp(-y_i(b + x_i^T w)) + 2\lambda w.$$

Then, noting that

$$\frac{\exp(-y_i(b + x_i^T w))}{1 + \exp(-y_i(b + x_i^T w))} = 1 - \mu_i(w, b),$$

we get that

$$\nabla_w J(w, b) = \frac{1}{n} \sum_{i=1}^n -y_i x_i (1 - \mu_i(w, b)) + 2\lambda w.$$

□

Claim: $\nabla_b J(w, b) = \frac{1}{n} \sum_{i=1}^n y_i (\mu_i(w, b) - 1)$.

Proof. Again using the chain rule and the equivalence $\frac{\exp(-y_i(b + x_i^T w))}{1 + \exp(-y_i(b + x_i^T w))} = 1 - \mu_i(w, b)$,

$$\begin{aligned} \nabla_b J(w, b) &= \frac{1}{n} \sum_{i=1}^n \frac{1}{1 + \exp(-y_i(b + x_i^T w))} \exp(-y_i(b + x_i^T w)) \cdot -y_i \\ &= \frac{1}{n} \sum_{i=1}^n -y_i (1 - \mu_i(w, b)) = \frac{1}{n} \sum_{i=1}^n y_i (\mu_i(w, b) - 1). \end{aligned}$$

□

- b. [8 points] Implement gradient descent with an initial iterate of all zeros. Try several values of step sizes to find one that appears to make convergence on the training set as fast as possible. Run until you feel you are near to convergence.

- (a) For both the training set and the test, plot $J(w, b)$ as a function of the iteration number (and show both curves on the same plot).

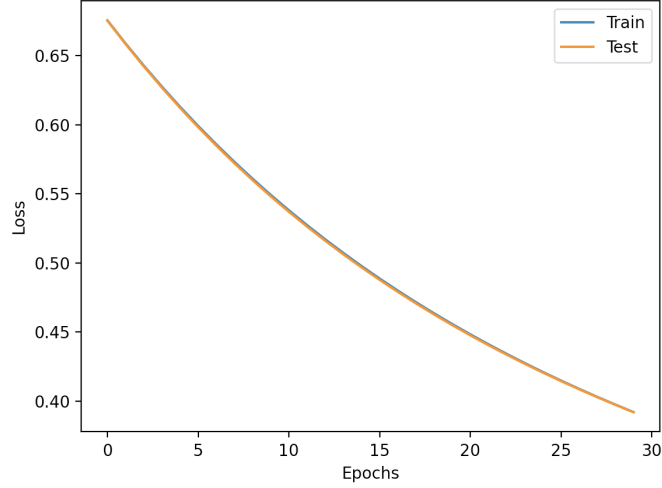


Figure 7: Part (b) Train and Test Loss as a function of Epochs (up to 30) for learning rate 1e-2 and (normal) gradient descent.

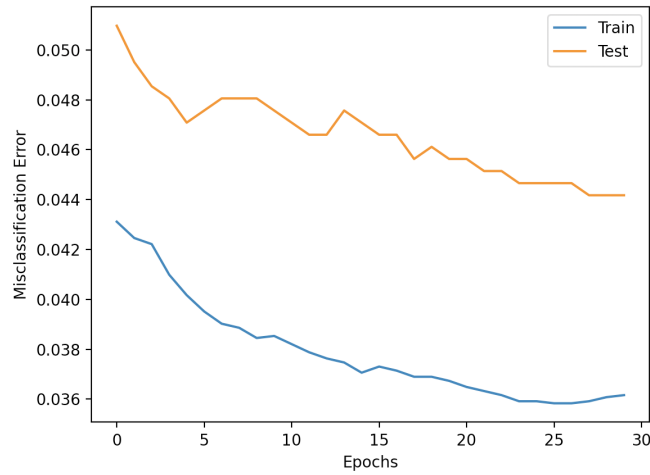


Figure 8: Part (b) Train and Test Misclassification error as a function of Epochs (up to 30) for learning rate 1e-2 and (normal) gradient descent.

- (b) For both the training set and the test, classify the points according to the rule $\text{sign}(b + x_i^T w)$ and plot the misclassification error as a function of the iteration number (and show both curves on the same plot).

Reminder: Make sure you are only using the test set for evaluation (not for training).

- c. [7 points] Repeat (b) using stochastic gradient descent with a batch size of 1. Note, the expected gradient with respect to the random selection should be equal to the gradient found in part (a). Show both plots described in (b) when using batch size 1. Take careful note of how to scale the learning rate.
- d. [7 points] Repeat (b) using stochastic gradient descent with batch size of 100. That is, instead of approximating the gradient with a single example, use 100. Note, the expected gradient with respect to the random selection should be equal to the gradient found in part (a).

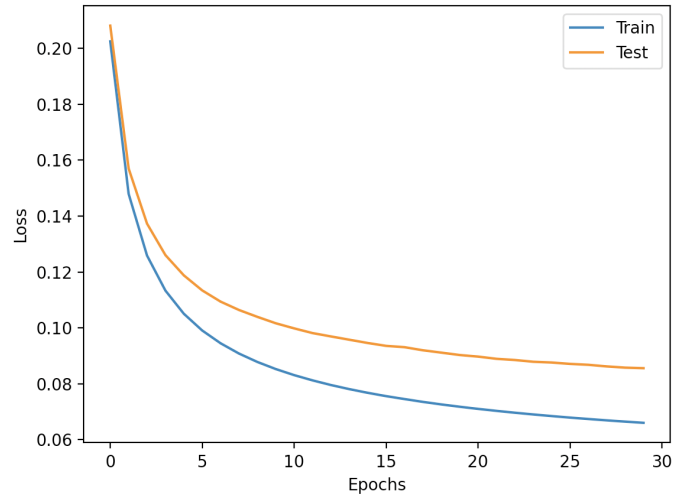


Figure 9: Part (c) Train and Test Loss as a function of Epochs (up to 30) for learning rate $1e-4$ and a batch size of 1.

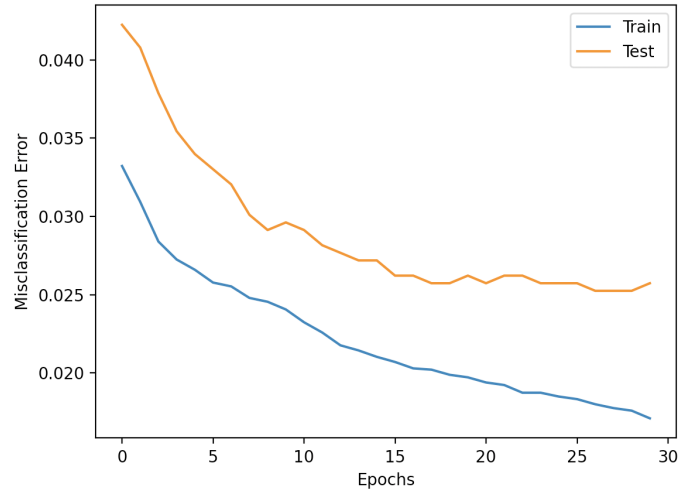


Figure 10: Part (c) Train and Test Misclassification error as a function of Epochs (up to 30) for learning rate $1e-4$ and batch size of 1.

What to Submit

- **Part a:** Proof
- **Part b:** Separate plots for b(i) and b(ii).
- **Part c:** Separate plots for c which reproduce those from b(i) and b(ii) for this case.
- **Part d:** Separate plots for c which reproduce those from b(i) and b(ii) for this case.
- **Code** on Gradescope through coding submission.

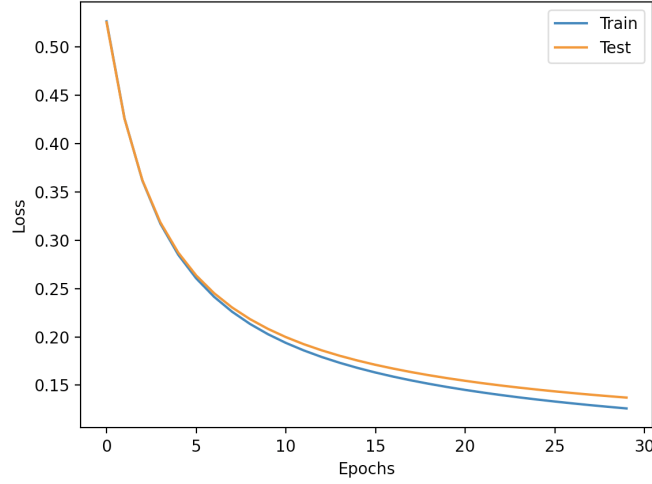


Figure 11: Part (d) Train and Test Loss as a function of Epochs (up to 30) for learning rate 1e-3 and batch size 100.

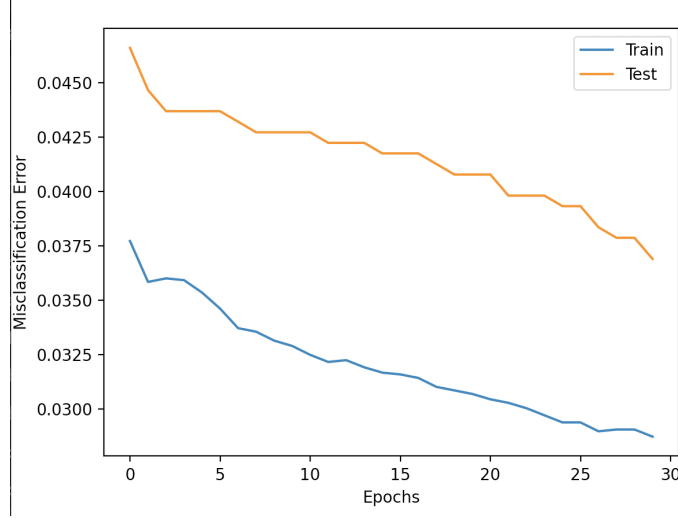


Figure 12: Part (d) Train and Test Misclassification error as a function of Epochs (up to 30) for learning rate 1e-3 and batch size 100.

Confidence Interval of Least Squares Estimation

Bounding the Estimate

B2. Let us consider the setting, where we have n inputs, $X_1, \dots, X_n \in \mathbb{R}^d$, and n observations $Y_i = \langle X_i, \beta^* \rangle + \epsilon_i$, for $i = 1, \dots, n$. Here, β^* is a ground truth vector in $\mathbb{R}^d$ that we are trying to estimate, the noise $\epsilon_i \sim \mathcal{N}(0, 1)$, and the n examples piled up — $X \in \mathbb{R}^{n \times d}$. To estimate, we use the least squares estimator $\hat{\beta} = \min_{\beta} \|X\beta - Y\|_2^2$. Moreover, we will use $n = 20000$ and $d = 10000$ in this problem.

- [3 points] Show that $\hat{\beta}_j \sim \mathcal{N}(\beta_j^*, (X^T X)^{-1}_{j,j})$ for each $j = 1, \dots, d$. (Hint: see [notes](#) on confidence intervals.)
- [4 points] Fix $\delta \in (0, 1)$ suppose $\beta^* = 0$. Applying the proposition from the notes, conclude that for

each $j \in [d]$, with probability at least $1 - \delta$, $|\hat{\beta}_j| \leq \sqrt{2(X^T X)_{j,j}^{-1} \log(2/\delta)}$. Can we conclude that with probability at least $1 - \delta$, $|\hat{\beta}_j| \leq \sqrt{2(X^T X)_{j,j}^{-1} \log(2/\delta)}$ for all $j \in [d]$ simultaneously? Why or why not?

- c. *[5 points]* Let's explore this question empirically. Assume data is generated as $x_i = \sqrt{(i \bmod d) + 1} \cdot e_{(i \bmod d) + 1}$ where e_i is the i th canonical vector and $i \bmod d$ is the remainder of i when divided by d . Generate each y_i according to the model above. Compute $\hat{\beta}$ and plot each $\hat{\beta}_j$ as a scatter plot with the x -axis as $j \in \{1, \dots, d\}$. Plot $\pm \sqrt{2(X^T X)_{j,j}^{-1} \log(2/\delta)}$ as the upper and lower confidence intervals with $1 - \delta = 0.95$. How many $\hat{\beta}_j$'s are outside the confidence interval? *Hint: Due to the special structure of how we generated x_i , we can compute $(X^T X)^{-1}$ analytically without computing an inverse explicitly.*

What to Submit:

- **Parts a, b:** Proof.
- **Part b:** Answer.
- **Part c:** Plots of $\hat{\beta}$ and its confidence interval **on the same plot**.

Administrative

A8.

- a. *[2 points]* About how many hours did you spend on this homework? There is no right or wrong answer :)

About 11 hours of timed work, not including all the time studying for the midterm that probably helped indirectly!